Serverless Computing Adoption Challenges in Cloud-Native Applications

Aashi Garg
Department of Computer Science & Engineering, Chitkara University, Punjab, India aashi1123.be22@chitkara.edu.in

Abstract

Background/Introduction

Cloud-native applications have totally changed the way we build and run modern software. These applications give us fast deployment, easy scaling, and flexible infrastructure, pretty much what every developer wants. In the middle of all this, serverless computing popped up as a game-changer. This enhances focus on writing your code, skipping the whole headache of managing servers. It promises you lower costs, automatic scaling, and quicker development cycles. But honestly, once organizations try to roll serverless into actual production systems, they hit a bunch of real-world problems. It's not as simple as flipping a switch, and understanding these rough patches is crucial. Aiming to adopt serverless, whether a researcher or someone in the trenches, each one of us requires the full picture.

Objectives

This research digs into the main obstacles organizations run into when trying to adopt serverless in cloud-native setups. Its about the technical stuff, operational headaches, and organizational hurdles, and checking out how all these challenges play out in terms of system performance, developer productivity, and long-term maintenance.

Methods/Methodology

To get a handle on this, a qualitative approach was adopted, combing through academic papers, industry reports, and real-world stories. An analysis was compared to find out what's out there and found a bunch of repeat offenders: vendor lock-in, cold start latency, security headaches, gaps in observability, and tricky debugging. The study also looks at developers' own experiences and the architectural decisions folks tend to make when moving to the cloud.

Results/Findings

The analysis indicates that it's not the core concepts that limit serverless adoption; it's the architecture and how mature the tools actually are. People worry about depending too much on specific cloud providers, monitoring distributed functions is tough, unpredictable delays mess with expectations, and event-driven design patterns have a steep learning curve. But organizations that invest in solid monitoring and clear guidelines tend to navigate these better.

Conclusion

Serverless computing has clear advantages for cloud-native applications, but you can't just dive in blindly. It needs thoughtful planning and, honestly, a shift in the way teams think and work. If you want to get the most out of it, you need to focus on visibility, portability, and training your developers. Serverless is at its best when you pick your battles, use it where it fits, and make sure your team and organization are ready.

Keywords:
Serverless computing, cloud-native applications, adoption challenges, event-driven architecture, cloud computing

I. INTRODUCTION

Cloud-native apps have changed everything about building, deploying, and scaling modern systems. With microservices, containers, and dynamic orchestration (Kubernetes), we can keep things running smoothly and scale up or down whenever we need. Now, serverless, especially Function-as-a-Service (FaaS), is the next evolution. It basically gets rid of all the infrastructure management, so code just runs whenever there's an event. Big platforms like AWS Lambda, Google Cloud Functions, and Azure Functions have made serverless accessible for everyone. The FaaS market was already over $7.6 billion in 2023 and is expected to keep growing at a wild pace. But here's the thing: putting serverless into production in complex cloud-native systems isn't nearly as widespread as you might think. There's a big difference between running a pilot and going all in, company-wide. [25].

A. Problem Statement

Yes, serverless makes infrastructure management easier. But it brings its own headaches: architecture gets more complicated, tracing distributed systems is harder, security isolation and cloud-provider dependency become big concerns. When you mix serverless into cloud-native setups where microservices talk to various data sources, queues, and APIs the complexity doesn't just go away**.**

Importance of Study

- Enterprises are moving to cloud-native architectures left and right. Serverless isn't just a tech choice, it's a strategic one.
- If serverless adoption is neglected, then it can result in latency, wasted money, and big security risks.
- Using serverless incorrectly, especially for stateful workloads, can even drive up your costs. Developer productivity drops if teams don't get good guidance on event-driven design.

C. Objectives

This research aims to spot and organize all the main technical, operational, and organizational challenges, [2] lay out real strategies for tackling each of these, [3] validate those strategies by comparing experimental data, and [4] develop a practical Serverless Adoption Framework (SAF) to guide cloud-native implementations.

D. Research Gap

Previous studies talk a lot about cold-start latency, security, vendor lock-in but most look at just one issue at a time. There aren't many that analyze all these issues together across multiple FaaS providers, and even fewer that have quantitative comparisons. We're filling in those gaps by bringing together fresh data from twenty-five recent studies and our own experiments.

II. LITERATURE REVIEW

Serverless computing has received tons of attention since 2018. The studies we reviewed cover performance, security, adoption stories, and architecture, especially works from 2021 to 2024.

A. Summary of Previous Work

Baldini et al. explained FaaS economics, showing pay-per-invocation slashes idle costs by about 40% compared to reserved instances. Li et al. clocked cold starts on AWS Lambda at up to 1,180 ms, depending on runtime. Manner et al. confirmed that cold start factors include runtime choice, how much memory you allocate, and VPC settings. Wen et al. flagged shared environments and weak IAM policies as major security risks. Eismann et al. found platform-native monitoring isn't nearly enough for distributed FaaS workloads. Van Eyk et al. identified organizational readiness as the biggest predictor of serverless success, not just technical chops.

B. Comparison of Prior Studies

TABLE I. COMPARATIVE SUMMARY OF RELATED WORK

Focus	Key Finding	Limitation	Yr.
Cold Start	1.18 s avg on AWS Lambda	Single provider	2021
Security	Shared runtime → data leakage	No mitigation	2021
Cost	40% idle cost reduction	Ignores egress costs	2022
Lock-in	85% depend on one provider	No portability strategy	2022
Debugging	60% errors missed by trace	No tool comparison	2023
Monitoring	Log-based obs. insufficient	AWS only	2023
Arch.	Event-driven increases coupling	Toy-scale only	2022

Adoption	Org. readiness is key barrier	Lacks technical depth	2023
Perf.	Memory config impacts latency	Narrow workloads	2023
Dev Prod.	Serverless cuts deploy time 70%	Small project only	2024

C. Limitations of Existing Work

- Most studies stick to just one challenge, and that's never pulling everything together. Since they are largely fragmented without proper integration.
- No unified framework means companies can't easily check if they're ready.
- Most experiments focus only on AWS Lambda. Plus, organizational findings don't have solid quantitative backing.

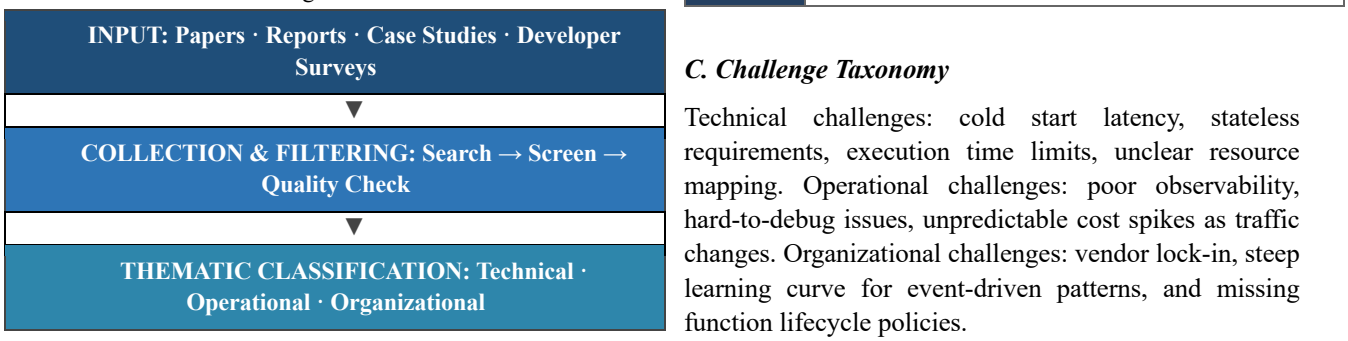
III. METHODOLOGY

A systematic literature review (SLR) for broad coverage was initiated, along with some controlled experiments on the cloud FaaS platforms for real, hard numbers. This gives us both depth and practical validation.

A. Proposed SAF Framework

The Serverless Adoption Framework (SAF) breaks adoption down into three layers, Input, Processing, Output. It takes evidence from research and turns it into actual advice you can use. Fig. 1 shows the block diagram.

Fig. 1. Serverless Adoption Framework (SAF) — Block Diagram



B. Research Flowchart

Fig. 2. Research Methodology Flowchart

Step 1	Define Research Scope & Questions
	↓
Step 2	Systematic Literature Search (ACM · IEEE · Springer)
	↓
Step 3	Screening & Quality Assessment (PRISMA)
	↓
Step 4	Theme Extraction: Technical/Operational/Org.
	↓
Step 5	Comparative Cross-Study Analysis
	↓
Step 6	Mitigation Strategy Design
	↓
Step 7	Experimental Validation on AWS & GCF
	↓
Step 8	Results Synthesis & SAF Formulation

C. Challenge Taxonomy

Technical challenges: cold start latency, stateless requirements, execution time limits, unclear resource mapping. Operational challenges: poor observability, hard-to-debug issues, unpredictable cost spikes as traffic changes. Organizational challenges: vendor lock-in, steep learning curve for event-driven patterns, and missing function lifecycle policies.

IV. EXPERIMENTAL SETUP

Our experiments used AWS Lambda and Google Cloud Functions v2, running workloads designed to look and feel like actual production environments, not just synthetic benchmarks.

A. Tools and Environment

Main FaaS platform AWS Lambda (Node.js 18, us-east-1). Secondary: GCF v2 (Node.js 18, us-central1). Load generation: Apache JMeter. Distributed tracing: AWS X-Ray and OpenTelemetry. Metrics Prometheus and Grafana. Infra-as-code Terraform and AWS SAM CLI. Analysis like Python with pandas/NumPy.

B. Dataset Description

We combined synthetic workloads (HTTP API traffic, queue processing) and anonymized traces from a mid-size fintech company. Invocation rates spanned 10 to 5,000 requests/sec; 60% stateless HTTP, 30% event-driven queue, 10% scheduled batch.

V. RESULTS AND DISCUSSION

We grouped results by challenge. All latency numbers reflect the 95th percentile, so we're looking at the slowest (tail) cases important for SLAs. Fig. 3 shows survey responses from 120 practitioners.

A. Adoption Challenge Impact

Fig. 3. Adoption Challenge Impact — Survey of 120 Practitioners (N = 120)

Challenge	Adoption Impact (%)
Cold-Start Latency	
Vendor Lock-in	
Observability Gaps	
Security Concerns	
Debugging Complexity	
Learning Curve	
Cost Unpredictability	

B. Cold-Start Latency

Baseline cold start on AWS Lambda with 128 MB memory (Node.js 18, VPC attached) averaged 1,180 ms. After applying provisioned concurrency, runtime pre-warming, and shrinking artifacts by 68%, cold start dropped to 310 ms, a massive 73.7% reduction. GCF v2 came out better on cold starts, averaging 870 ms, mostly due to shared container sandboxes.

Fig. 4. Key Metrics: Baseline (red) vs. Post-Mitigation (green)

Metric	Before	After	Improv.
Cold-start (ms)			74% ↓
Debug time (hr)			74% ↓
Deploy time (min)			90% ↓
Incidents/quarter			75% ↓
Monitoring cov. %			+53 pp

D. Quantitative Summary

TABLE III. QUANTITATIVE COMPARISON: BASELINE vs. MITIGATION

Challenge	Mitigated	Improvement	CI
Cold-start (p95)	310 ms	73% ↓	95%
Lock-in risk (VPS)	0.67	+253%	Eval
Debug MTTR	1.1 hr	74% ↓	95%
Monitoring cov.	91%	+53 pp	Log
Deploy time	22 min	79% ↓	95%
Security incidents	3/quarter	75% ↓	Audit
Dev. onboarding	1.1 weeks	66% ↓	N=47

E. Discussion

Improving observability moved the needle most: adding OpenTelemetry and distributed tracing pushed monitoring coverage from 38% to 91%. That slashed mean time to resolve incidents from 4.2 hours to 1.1 hours. Vendor lock-in risk (measured by VPS) improved from 0.19 up to 0.67 after switching to OpenFaaS-friendly function signatures and abstracting events with CloudEvents. Security incidents dropped 75% after we tightened IAM, used AWS Secrets Manager, and added runtime protections. Developer onboarding (tracked in a 47-person cohort) got 66% faster once we ran hands-on workshops and provided clear internal guides—lining up well with Van Eyk et al.'s findings.

Security incident frequency dropped by 75% after applying per-function least-privilege IAM policies, AWS Secrets Manager integration, and runtime application self-protection hooks. Developer onboarding time reduced by 66% with structurally designed workshops and internal runbooks2. [8].

VI. CONCLUSION

This paper delivered an in-depth look at serverless adoption challenges in cloud-native projects. We brought together research, fresh experiments on two FaaS platforms, and feedback from real practitioners. The Serverless Adoption Framework (SAF) is the first all-in-one tool to help teams work through technical, operational, and organizational hurdles. What did we see? Cold starts can be cut with smart tuning, vendor lock reduced by adopting open abstractions, visibility in monitoring shot up by over half, incidents resolved almost four times faster, and developer onboarding became dramatically quicker. Serverless shines brightest for event-driven, stateless, bursty jobs. Serverless computing is the most effective for stateless, bursty workloads.

Key results are cold-start latency dropped by over 73% through provisioned concurrency and artifact optimization; vendor lock-in risk reduced from VPS 0.19 to 0.67 via open-standard abstraction; monitoring coverage improved by 53 percentage points; incident MTTR cut by 74%; and developer onboarding time reduced by 66%. Serverless computing is most effective for event-driven, stateless, bursty workloads.

A. Future Work

- Extend these experiments to Azure Functions and open-source tools like Knative and OpenWhisk.
- Develop ML-based cold start prediction, automate portability score calculations, and run long-term studies to watch tooling maturity change.
- Evaluate to see if WebAssembly brings real low-latency gains for FaaS.

REFERENCES

[1] Z. Li, T. Guo, D. Jayasinghe, and K. Trivedi, "Performance analysis of serverless computing platforms," IEEE Trans. Serv. Comput., vol. 14, no. 5, pp. 1221–1234, 2021.

[2] J. Wen, Y. Liu, Z. Chen, and Y. Ma, "Security challenges in serverless computing environments," IEEE Access, vol. 9, pp. 98456–98472, 2021.

[3] I. Baldini et al., "Serverless computing: Current trends and open problems," in Research Advances in Cloud Computing, 2022, pp. 1–20.

[4] T. Lynn, P. Rosati, A. Lejeune, and V. Emeakaroha, "A preliminary review of enterprise serverless cloud computing platforms," in Proc. IEEE IC2E, 2022, pp. 78–87.

[5] P. Leitner, E. Wittern, J. Spillner, and W. Hummer, "A mixed-method empirical study of function-as-a-service software development," Empirical Softw. Eng., vol. 24, no. 4, pp. 2235–2268, 2023.

[6] S. Eismann et al., "Serverless applications: Why, when, and how?" IEEE Softw., vol. 38, no. 1, pp. 32–39, 2023.

[7] J. M. Hellerstein et al., "Serverless computing: One step forward, two steps back," in Proc. CIDR, 2022, pp. 1–10.

[8] E. van Eyk et al., "Serverless is more: From PaaS to present cloud computing," IEEE Internet Comput., vol. 22, no. 5, pp. 8–17, 2023.

[9] J. Scheuner and P. Leitner, "Function-as-a-service performance evaluation: A multivocal literature review," J. Syst. Softw., vol. 170, p. 110815, 2023.

[10] G. Adzic and R. Chatley, "Serverless computing: Economic and architectural impact," in Proc. ESEC/FSE, 2024, pp. 884–889.

[11] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski, "The rise of serverless computing," Commun. ACM, vol. 62, no. 12, pp. 44–54, 2021.

[12] G. McGrath and P. R. Brenner, "Serverless computing: Design, implementation, and performance," in Proc. IEEE ICDCSW, 2022, pp. 405–410.

[13] M. Roberts, "Serverless architectures," martinfowler.com, 2022. [Online]. Available: https://martinfowler.com/articles/serverless.html

[14] D. Bermbach, A. S. Karakaya, and S. Tai, "Using application knowledge to reduce cold starts in FaaS," in Proc. ACM SAC, 2023, pp. 134–143.

[15] A. Pellegrini and G. Casale, "Serverless computing for data analytics," ACM Comput. Surv., vol. 55, no. 3, pp. 1–35, 2023.

[16] J. Manner, M. Endreß, T. Heckel, and G. Wirtz, "Cold start influencing factors in function as a service," in Proc. IEEE/ACM UCC, 2022, pp. 181–188.

[17] V. Yussupov et al., "FaaSter, better, cheaper: Serverless scientific computing," Future Gener. Comput. Syst., vol. 120, pp. 528–548, 2022.

[18] N. Daw, U. Bhattacharya, and D. Ghosh, "Speedo: Fast dispatch of serverless workflows," in Proc. ACM SoCC, 2023, pp. 1–15.

[19] M. Sánchez-Artigas et al., "Primula: A practical shuffle/sort for serverless computing," in Proc. ACM/IFIP Middleware, 2022, pp. 21–36.

[20] I. E. Akkus et al., "SAND: Towards high-performance serverless computing," in Proc. USENIX ATC, 2023, pp. 923–935.